자료구조와 알고리즘

학습목표

- 자료구조에 대해서 이해하고 적절하게 사용할 수 있다.
- 기초 알고리즘을 이해하고 적절하게 사용할 수 있다.
- 알고리즘을 분석할 수 있다.
- 프로그래밍 능력을 증진시킨다.

개요

컴퓨터 프로그램은 데이터를 입력받아 조작하고 반환하는 것이 전부다. 데이터를 어떻게 조직하냐에 따라 프로그램은 수십 수백 배 더 빠르거나 느리게 실행될 수 있다. 이러한 데이터를 조직하는 방법을 **자료구조**(data structure)라고 하는데, 더 나은 프로그램을 작성하기 위해서는 다양한 자료구조를 알고, 각각의 자료 구조가 성능에 어떤 영향을 미치는지 알고 있어야 한다. 또한 적절한 자료구조를 사용하는 것뿐만 아니라, 데이터 조작에 적합한 **알고리즘**(algorithm)을 선택하는 것도 중요하다. 이번 강의에서는 C++에서 제공하는 STL 컨테이너을 직접 구현해보며 자료구조에 대한 이해도를 올리고, 그 외 유용한 알고리즘을 배워본다.

자료구조

자료구조는 크게 **선형 구조**(Linear Structure)와 **비선형 구조**(Non-linear Structure)로 나눌 수 있으며, 선형 구조에는 리스트, 스택, 큐가 있고, 비선형 구조에는 그래프와 트리가 있다. 또, 대부분의 자료구조는 네 가지 기본 방법을 사용하며 이를 **연산**(Operation)이라고 한다.

- 읽기 : 자료구조 내 특정 위치를 찾아보는 것이다.
- 검색 : 자료구조 내 특정 값을 찾는 것이다.
- 삽입 : 자료구조에 새로운 값을 추가하는 것이다.
- 삭제 : 자료구조 내 특정 값을 삭제하는 것이다.

자료구조를 구현하는 방법에는 크게 **순차 자료구조**(contiguous data structure)와 **연결 자료구조**(linked data structure)가 있는데, 순차 자료구조는 구현할 자료들을 논리적인 순서대로 메모리에 **연속**하여 저장하는 구현 방식이고, 연결 자료구조는 노드라는 여러 개의 메모리 청크에

데이터를 저장하며, 이를 **연결**하여 구현하는 방식이다. 즉, 순차 자료구조는 배열을 이용하고, 연결 자료구조는 포인터를 이용한다. 둘을 비교하면 아래와 같다.

순차 자료구조	연결 자료구조
모든 데이터가 메모리에 연속적으로 저장된다.	데이터는 노드에 저장되고, 노드는 메모리 곳곳에 흩어져 있을 수 있다.
임의 원소에 즉각적으로 접근할 수 있다.	임의 원소에 접근하는 것은 선형 시간 복잡도를 가지며 느린 편이다.
캐시 지역성* 효과로 인해 모든 데이터를 순회하는 것이 매우 빠르다.	캐시 지역성* 효과가 없어 모든 데이터를 순회하는 것이 느리다.
데이터 저장을 위해 정확하게 데이터 크기만큼 메모리를 사용한다.	각 노드에서 포인터 저장을 위해 여분의 메모리를 사용한다.

*지역성(locality) : CPU가 기억장치의 특정 부분에 위치한 데이터나 프로그램 코드를 집중적으로 액세스하는 현상이다.

프로그래머는 둘의 특징을 명확히 알고 구현할 작업의 요구 조건 및 사용 빈도에 따라 둘 중 하나를 선택하거나 또는 두 개를 조합하여 응용 프로그램을 개발해야 한다.

알고리즘의 분석

알고리즘이란 문제를 해결하기 위한 절차를 말한다. 문제를 해결하는 방법에는 여러가지가 존재할 수 있으며, 우리는 이 중 최선을 선택해야 한다. 다시 말해 알고리즘을 분석하고 비교할 줄 알아야 한다. 알고리즘을 분석하는 것은 알고리즘의 **자원**(resource)*의 사용량을 분석하는 것이라고 볼 수 있는데, 자원을 적게 사용할 수록 효율적인 알고리즘이라고 할 수 있다.

*자원이란 실행 시간, 메모리 사용량 등을 말한다.

시간 복잡도

알고리즘의 분석에 있어 가장 중요한 부분은 실행 시간이다. 알고리즘의 실제 실행은 하드웨어 및 소프트웨어 환경에 따라 천차만별이므로 단순 측정으로는 실행 시간을 분석할 수 없다. 다시 말해 하드웨어와 소프트웨어 환경을 배제한 객관적인 지표가 필요하다. 이를 위해서 시간 복잡도를 사용한다.

시간 복잡도(time complexity)는 알고리즘을 수행하는 데 필요한 연산이 몇 번 실행되는지를 숫자로 표기한다. 이 연산의 개수는 상수가 아니라 입력한 데이터의 개수를 나타내는 n에 따라 변하게 된다. 그래서 연산의 개수 n의 값에 따라 시간 복잡도를 나타낸 것을 시간 복잡도 함수라고

하며 수식으로는 $T(n)$이라고 표기한다. 일례로 양의 정수 n을 n번 더하는 알고리즘을 구현한다고 할 때, 이에 따른 시간 복잡도 함수를 확인해보자.

```
// Case 1
let sum = n * n;

// Case 2
let sum = 0;
for (let i = 0; i < n; ++i)
   sum += n;

// Case 3
let sum = 0;
for (let i = 0; i < n; ++i)
   for (let j = 0; j < n; ++j)
      sum += 1;
```

위 3개의 알고리즘의 연산 횟수를 비교해보면 아래와 같이 나타낼 수 있다. (간단히 나타내기 위해 루프 제어 연산은 제외한다.)

연산 종류	Case 1	Case 2	Case 3
대입 연산	1	$n + 1$	$n^2 + 1$
덧셈 연산		n	n^2
곱셈 연산	1		
전체 연산 횟수	2	$2n + 1$	$2n^2 + 1$

하나의 연산이 t만큼의 시간이 필요하다고 하면 이에 대한 시간 복잡도 함수를 각각 $2t$, $(2n + 1)t$, $(2n^2 + 1)t$로 나타낼 수 있다.

시간 복잡도를 구했으므로 알고리즘의 수행 시간을 측정할 수 있게 됐다. 위의 예시를 바탕으로 입력값에 따라 어느정도의 시간이 걸리는 지를 계산해보자. 편의상 모든 연산에 걸리는 시간은 1ms라고 가정한다.

입력값	Case 1	Case 2	Case 3
$n = 1$	2ms	3ms	3ms
$n = 10$	2ms	21ms	201ms
$n = 100$	2ms	201ms	20,001ms = 약 20s
$n = 1000$	2ms	2,001ms = 약 2s	2,000,001ms = 약 33m
$n = 10000$	2ms	20,001ms = 약 20s	200,000,001ms = 약 55h

입력값이 작을 때는 그 차이가 크지 않았지만, 입력값이 점점 커질 수록 엄청나게 차이가 벌어진다는 것을 볼 수 있다.* 그러므로 우리는 알고리즘을 효율적으로 구현하도록 늘 고민해야 한다.

*이를 실행 시간의 **성장률**(rate of growth)이라고 한다.

Big-O 표기법

앞의 결과값을 다시 한번 관찰해보자. 다항식의 시간 복잡도 함수에서 입력값이 커져감에 따라 각 항의 결과값에 관여하는 정도가 달라지게 됨을 알 수 있다. 일례로 Case3의 시간 복잡도 함수 $T(n) = 2n^2 + 1$에서 입력값을 10000을 넣었을 때, 각 항의 계산값은 $2(10,000)^2 = 200,000,000$과 1 이므로 $2n^2$대비 1의 영향이 매우 작음을 알 수 있다.

그래서 시간 복잡도와 뒤에 배울 공간 복잡도를 표시할 때는 각 함수의 모든 항을 표시하지 않으며 상수 계수와 중요하지 않은 항목을 제거한 점근적 표기법(asymptotic notation)을 사용한다. 여기에는 Big-θ, Big-O, Big-Ω 등이 있지만 이중 가장 많이 사용되는 것은 Big-O 표기법이다. Big-O 표기법은 점근적 상한선을 제공한다.

앞선 시간 복잡도 함수를 Big-O 표기법으로 나타내면 각각 $O(1)$, $O(n)$, $O(n^2)$으로 나타낼 수 있다. 많이 쓰이는 Big-O 표기법은 아래와 같으며 오른쪽으로 갈 수록 상한선이 높아진다. 다시 말해 왼쪽에 위치한 시간 복잡도일수록 해당 알고리즘은 빠르다고 할 수 있다.

$$O(1) < O(logn) < O(n) < O(nlogn) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

유의할 점은 한 알고리즘이 다른 알고리즘보다 실제로 빠르다고 하더라도 같은 방식으로 표현이 될 수도 있다는 것이다. 따라서 같은 카테고리라면 어떤 알고리즘이 더 빠른지 면밀한 분석이 필요할 때가 있다.

공간 복잡도

알고리즘의 성능을 측정할 때는 실행 시간만 따지지 않는다. 컴퓨터 내의 자원 공간을 얼마나 사용하는지도 따져봐야 한다. **공간 복잡도**(space complexity)는 알고리즘을 수행하는 데 필요한 자원 공간의 양을 말한다. 공간 복잡도 함수는 고정 공간* 요구 + 가변 공간** 요구로 나타낼 수 있으며 수식으로는 $S(P) = c + S_p(n)$으로 표기한다.

*고정 공간 : 입력과 출력의 횟수나 크기와 관계 없는 공간

**가변 공간 : 문제를 해결하기 위해 요구되는 추가 공간

팩토리얼의 구현을 통해 공간 복잡도를 살펴보자.

```
function factorial(n) {
  if (n <= 1) {
    return 1;
  }

  return n * factorial(n - 1);
}

function factorial(n) {
  let sum = 1;
  for (let i = n; i > 1; --i) {
    sum *= i;
  }

  return sum;
}
```

재귀적으로 구현한 함수의 공간 복잡도는 $O(n)$이며, 아래는 $O(1)$이다.

시간 복잡도 vs 공간 복잡도

여태까지 배운 내용을 토대로 정리하자면 효율적인 알고리즘은 실행 시간이 적게 걸리며, 자원을 적게 소모하는 것이라고 할 수 있다. 하지만 시간과 공간은 반비례적인 경향이 있어 보통 알고리즘의 척도는 시간 복잡도를 위주로 판단한다. 즉, 시간 복잡도를 위해 공간 복잡도를 희생하는 경우가 많다.

STL

대부분의 언어는 프로그래머의 편의성을 위해 여러 자료구조와 알고리즘을 제공하고 있다. C++에서는 템플릿을 이용해 제공하고 있으며 이를 **표준 템플릿 라이브러리**(Standard Template Library)라고 한다. STL은 여러 파트로 구성되어 있지만 그 중 우리가 살펴볼 파트는 아래와 같다.

- 임의 타입의 객체를 보관할 수 있는 컨테이너 라이브러리(Container Library)
- 컨테이너에 보관된 원소에 접근할 수 있는 반복자 라이브러리(Iterator Library)
- 반복자를 가지고 일련의 작업을 수행하는 알고리즘 라이브러리(Algorithm Library)

컨테이너는 앞으로 배우게 될 자료구조의 구현체이며, 반복자는 컨테이너의 요소에 접근할 수 있는 포인터다. 이렇게 나뉘게 된 이유는 구현 코드의 개수를 줄이기 위함이다. 컨테이너의 개수를 N, 알고리즘의 개수를 M이라 한다면, 컨테이너 각각에 알고리즘을 구현할 때 필요한 구현 코드의 개수는 N * M이다. 이 사이에 반복자를 추가함으로써 반복자를 통해 각 요소에 추상적으로 접근이 가능하게 돼 총 필요한 구현 코드의 개수를 N + M으로 줄일 수 있었다.*

* 이를 반복자 패턴(Iterator Pattern)이라고 한다.

리스트

리스트(list)는 순서를 갖고 있는 자료구조다. 순차 자료구조엔 선형 리스트가 있고, 연결 자료구조엔 단일 연결 리스트, 원형 연결 리스트, 이중 연결 리스트가 있다.

선형 리스트

선형 리스트(Linear List)는 순서 리스트(Ordered List)라고도 하며 STL 상에서는 std::array, std::vector로 구현되어 있다. 임의 접근이 가능하다는 특징을 가지고 있다.

- **읽기**

선형 리스트는 임의 접근이 가능하기 때문에 $O(1)$의 시간이 걸린다.

- **검색**

하나하나 원소를 비교해가야 하므로 $O(n)$의 시간이 걸린다. 정렬되어 있다면 이진 검색*을 사용할 수 있으며 이 경우 $O(logn)$이다.

* 이진 검색은 추후 배운다.

- **삽입**

원소를 어디에 삽입하냐에 따라 시간이 달라진다. 맨 끝에 데이터를 추가할 경우 $O(1)$이지만, 처음이나 중간이라면 모든 데이터를 이동해야 하기에 $O(n)$이 걸린다.

- **삭제**

삽입과 마찬가지로 원소를 어디에서 삭제하냐에 따라 시간이 달라진다. 맨 끝의 데이터를 삭제할 경우 $O(1)$이지만, 처음이나 중간이라면 모든 데이터를 이동해야 하기에 $O(n)$이 걸린다.

std::array

std::array는 정적 배열로 크기가 정해져 있으며, 기존 배열과 사용이 유사하다.

```
#include <array> // std::array를 위한 헤더
#include <iostream>

int main()
{
    std::array<int, 5> arr; // int arr[5];
    std::array<int, 5> arr2 = { 1, 2, 3, 4, 5 }; // 초기화는 기존 배열과
비슷하다.

    // 삽입, 삭제와 관련된 연산은 따로 존재하지 않는다.

    // 읽기는 아래와 같은 연산을 사용할 수 있다.
    std::cout << "arr2[1] : " << arr2[1] << "\n"; // [] 연산자
    std::cout << "arr2.front() : " << arr2.front() << "\n"; //
front() : 첫 번째 원소
    std::cout << "arr2.back() : " << arr2.back() << "\n"; // back() :
마지막 원소

    // 배열의 크기를 확인할 수 있다.
    std::cout << "arr2.size() : " << arr2.size() << "\n";

    // 비교도 가능하다.
    arr = arr2;
    if (arr == arr2)
    {
```

```
        std::cout << "arr is same as arr2\n";
    }

    // C++17부터는 추론이 된다.
    std::array arr3 = { 1, 2, 3 };

    // C++20부터는 to_array가 제공된다.
    auto arr4 = std::to_array("foo"); // arr4는 std::array<char, 4> {
'f', 'o', 'o', '\0' }

    return 0;
}
```

std::vector

std::vector는 동적 배열로 사용하다 크기가 부족하면 자동으로 크기를 늘린다. 늘어나는 크기는 컴파일러마다 다르나 통상적으로 약 1.5배에서 2배이다.

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    // 벡터 생성하기
    vector<int> vec;
    cout << boolalpha << vec.empty() << endl;
    cout << vec.size() << endl;
    cout << vec.capacity() << endl;

    // 삽입
    // push_back : 벡터의 맨 끝에 데이터 삽입
    vec.push_back(3);
    vec.push_back(4);
    vec.push_back(2);
    // { 3, 4, 2 }

    // 반복자
    // begin() : 컨테이너의 첫 원소를 가리키는 반복자
    // end() : 컨테이너의 마지막 원소의 다음을 가리키는 반복자
```

```
    for (vector<int>::iterator iter = vec.begin(); iter != vec.end();
++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // rbegin() : 컨테이너의 마지막 원소를 가리키는 반복자
    // rend() : 컨테이너의 첫 원소의 이전을 가리키는 반복자
    for (vector<int>::reverse_iterator iter = vec.rbegin(); iter !=
vec.rend(); ++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // insert : pos 이전에 데이터를 삽입
    vector<int>::iterator iter = vec.begin();
    ++iter; // 2번째 원소 : 4

    iter = vec.insert(iter, 10); // vec : { 3, 10, 4, 2 }, iter :
&vec[1] (10)
    iter = vec.insert(iter, 3, 5); // vec : { 3, 5, 5, 5, 10, 4, 2 },
iter : &vec[1] (5)
    iter = vec.insert(iter, vec.begin() + 3, vec.end() - 1); // vec :
{ 3, 5, 10, 4, 5, 5, 5, 10, 4, 2 }
                                        // iter : &vec[1] (5)
    for (vector<int>::iterator iter = vec.begin(); iter != vec.end();
++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // 삭제
    // pop_back : 맨 끝에 있는 데이터를 삭제
    // vec : { 3, 5, 10, 4, 5, 5, 5, 10, 4, 2 }
    vec.pop_back();
    vec.pop_back();
    vec.pop_back();
    // vec : { 3, 5, 10, 4, 5, 5, 5 }
```

```
    // erase : pos에 있는 데이터를 삭제
    vec.erase(vec.begin());
    // vec : { 5, 10, 4, 5, 5, 5 }
    vec.erase(vec.begin() + 1, vec.begin() + 3);
    // vec : { 5, 5, 5, 5 }

    // 읽기
    cout << vec.front() << endl; // 5
    cout << vec.back() << endl; // 5
    cout << vec[2] << endl; // 5

    // 검색 => 메소드가 아니라 알고리즘 라이브러리에 존재

    // 다른 생성
    vector<int> vec2(5); // { 0, 0, 0, 0, 0 }
    vector<int> vec3(5, 10); // { 10, 10, 10, 10, 10 }

    return 0;
}
```

std::vector 구현

동적으로 저장 공간이 늘어나는 std::vector를 구현해보자.

```
class MyVector
{
public:
  // 기본 생성자
  MyVector();

  // count만큼 공간이 할당된 생성자
  explicit MyVector(size_t count);

  // 복사 생성자. 깊은 복사(deep copy)가 이뤄져야 한다.
  MyVector(const MyVector& other);

  // 할당 연산자. 깊은 복사(deep copy)가 이뤄져야 한다.
```

```
MyVector& operator=(const MyVector& rhs);

// 소멸자
~MyVector();

// 첫 번째 요소를 가리키는 반복자를 반환한다.
int*            begin();
const int*      begin() const;

// 마지막 요소의 다음 번째를 가리키는 반복자를 반환한다.
int*            end();
const int*      end() const;

// 컨테이너가 비었는지 검사한다.
bool            empty() const;

// 원소의 개수를 반환한다.
size_t          size() const;

// 현재 할당된 공간의 크기를 반환한다.
size_t          capacity() const;

// pos에 위치한 원소의 참조를 반환한다. 만약 pos가 범위에서 벗어나면 std::out_of_range
예외가 던져진다.
int&            at(size_t pos);
const int&      at(size_t pos) const;

// pos에 위치한 원소의 참조를 반환한다.
int&            operator[](size_t pos);
const int&      operator[](size_t pos) const;

// 컨테이너의 첫 번째 원소의 참조를 반환한다.
```

```
    int&            front();
    const int&      front() const;

    // 컨테이너의 마지막 원소의 참조를 반환한다.
    int&            back();
    const int&      back() const;

    // 컨테이너를 비운다.
    void            clear();

    // pos 이전 위치에 value를 삽입한다.
    // value가 삽입된 곳을 가리키는 포인터를 반환한다.
    int*            insert(int* pos, int value);

    // pos에 위치한 원소를 지운다.
    // 삭제된 요소의 다음 포인터를 반환한다.
    int*            erase(int* pos);

    // 컨테이너의 맨 끝에 원소를 추가한다.
    void            push_back(int value);

    // 컨테이너의 마지막 원소를 삭제한다.
    void            pop_back();

    // value가 존재하는지 검사한다.
    bool            contains(int value);

    // 컨테이너의 용량을 newCapacity보다 같거나 크게 늘린다.
    // newCapacity > _capacity라면 새로운 메모리를 할당하고,
    // 그렇지 않다면 아무 동작도 수행하지 않는다.
    void            reserve(size_t newCapacity);
private:
```

```
    int*            _container;
    size_t          _size;
    size_t          _capacity;
};
```

실습해보기

1. MyVector를 템플릿으로 구현해보세요.
2. 조건에 따라 MyVector를 구현해보세요.

```
//-----------------------------------------------------------------------------------------
// 벡터 (vector) 클래스 구현하기
//
// 본 header 파일에서는, 신입 입사지원자의 과제용으로, 간단한 벡터를 구현하기 위한,
// MyVector 라는 클래스를 정의하고 있습니다.
//
// 소스와 주석을 참고하여, 필요한 기능들을 구현하여,
// MyVector.h 파일과 MyVector.cpp, 2개의 파일을 제출해주시면 됩니다.
// (제출하시는 파일들의 인코딩을 utf-8 로 맞춰주시면 감사하겠습니다.)
//
// 주의: std::vector 와 같은, 이미 구현된 벡터를 감싸는식으로 구현하지 마시고,
//    직접 자신의 코드로 기능을 구현 부탁드립니다.
//
// 참고: 영어로 작성된 부분이 많은 것에 대한 설명을 드리자면,
//    영어 실력을 중요하게 여긴다기 보다는, 번역기를 돌리던 관련 한글 문서를 찾던,
//    어떻게 해서든 원서의 내용을, 이해/학습하는 능력이 중요하기 때문입니다.
//-----------------------------------------------------------------------------------------



//-----------------------------------------------------------------------------------------
// 벡터란? (http://www.cplusplus.com/reference/vector/vector/)
//
```

```
// Vectors are sequence containers representing arrays that can change in size.
// Just like arrays, vectors use contiguous storage locations for their elements,
// which means that their elements can also be accessed using offsets on regular pointers
// to its elements, and just as efficiently as in arrays. But unlike arrays,
// their size can change dynamically, with their storage being handled automatically
// by the container.
//
// Internally, vectors use a dynamically allocated array to store their elements.
// This array may need to be reallocated in order to grow in size when new elements are
inserted,
// which implies allocating a new array and moving all elements to it.
// This is a relatively expensive task in terms of processing time, and thus,
// vectors do not reallocate each time an element is added to the container.
//
// Instead, vector containers may allocate some extra storage to accommodate for possible
growth,
// and thus the container may have an actual capacity greater than the storage strictly
needed to
// contain its elements (i.e., its size). Libraries can implement different strategies for
growth
// to balance between memory usage and reallocations, but in any case,
// reallocations should only happen at logarithmically growing intervals of size
// so that the insertion of individual elements at the end of the vector can be provided with
// amortized constant time complexity.
//-----------------------------------------------------------------------------------------------

#include <string>

//-----------------------------------------------------------------------------------------------
// MyVector 가 관리하는 오브젝트
//-----------------------------------------------------------------------------------------------
struct MyObject
```

```
{
    int _id;
};

//-----------------------------------------------------------------------------------------------
// MyVector 클래스.
//-----------------------------------------------------------------------------------------------
class MyVector
{
private: // 구현에 필요한 멤버 추가 함수/변수들을 자유롭게 아래에 정의 합니다.

    // 예.1) 사이즈를 저장하는 멤버변수를 아래처럼 추가하면 됩니다.
    // int _vectorSize;

    // 예.2) 저장공간을 늘리는 함수를 아래처럼 추가하면 됩니다.
    // void Grow();

public: // 생성자, 복사생성자, 할당연산자, 소멸자를 .cpp 파일에 구현합니다.

    // Constructor.
    MyVector(int capacity);

    // Copy constructor.
    MyVector(const MyVector& other);

    // Assignment operator.
    MyVector& operator=(const MyVector& other);

    // Destructor.
    ~MyVector();

public: // 아래 기능 함수들을 .cpp 파일에 구현합니다.
```

```
// Returns current capacity of this vector.
int GetCapacity() const;

// Returns current size of this vector.
int GetSize() const;

// Creates a new MyObject instance with the given ID, and appends it to the end of this
vector.
void Add(int id);

// Returns the first occurrence of MyObject instance with the given ID.
// Returns nullptr if not found.
MyObject* FindById(int MyObjectId) const;

// Trims the capacity of this vector to current size.
void TrimToSize();

// Returns the MyObject instance at the specified index.
MyObject& operator[](size_t index);

// Returns string representation of the vector.
std::string ToString() const;

// Remove all MyObject instances with the given ID in this vector.
void RemoveAll(int MyObjectId);

// Returns a newly allocated array of MyVector objects,
// each of whose elements have the same "_id" value of the MyObject struct.
// The 'numGroups' is an out parameter, and its value should be set to
// the size of the MyVector array to be returned.
MyVector* GroupById(int* numGroups);
```

```
};
```

3. 아래 문제를 풀어보세요.
    a. 코딩테스트 연습 - 모의고사

이진 검색

이진 검색(Binary Search)은 검색할 범위를 절반으로 줄여가며 검색하는 방법이다. 이진 검색은 정렬된 선형 리스트에서 사용할 수 있다. 아래는 C++로 구현한 이진 검색 코드다.

```
// arr : 정렬된 배열
// size : 배열의 크기
// value : 찾고자 하는 값
bool BinarySearch(int* arr, int size, int value)
{
    // 범위는 [s, e)로 표현한다. 구현에 따라 다르게 할 수 있다.
    int s = 0, e = size;
    while (s < e)
    {
        int m = (s + e) / 2;
        // 오버플로우를 방지하기 위해 아래처럼도 사용할 수 있다.
        // int m = s + (e - s) / 2;

        // 찾았다면 true다.
        if (arr[m] == value)
            return true;
        // 배열이 정렬되어 있기 때문에
        // 중간값과 비교하여 범위를 조정한다.
        else if (arr[m] < value)
            s = m + 1;
        else
            e = m;
    }

    return false;
}
```

이진 검색은 STL에서도 제공하는데, 목록은 여기를 참고한다.

```
#include <algorithm>
#include <iostream>

int main()
{
    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    // std::binary_search는 이진 검색으로 주어진 값을 찾는다.
    if (std::binary_search(arr, arr + 10, 3))
    {
        std::cout << "arr에는 3이 존재한다.\n";
    }

    int arr2[] = { 1, 2, 4, 5, 5, 6 };

    // std::lower_bound는 이진 검색으로
    // element >= value의 조건을 만족하는 값을 찾는다.
    std::cout << "\n[lower_bound]\n";
    for (int i = 0; i < 8; ++i)
    {
        int* lowerBound = std::lower_bound(arr2, arr2 + 6, i);

        std::cout << i << " <= ";
        if (lowerBound == arr2 + 6)
        {
            std::cout << "not found\n";
        }
        else
        {
            std::cout << *lowerBound << " at index " << lowerBound -
arr2 << "\n";
        }
    }

    // std::upper_bound는 이진 검색으로
    // element < value의 조건을 만족하는 값을 찾는다.
    std::cout << "\n[upper_bound]\n";
    for (int i = 0; i < 8; ++i)
    {
        int* upperBound = std::upper_bound(arr2, arr2 + 6, i);

        std::cout << i << " < ";
```

```
        if (upperBound == arr2 + 6)
        {
            std::cout << "not found\n";
        }
        else
        {
            std::cout << *upperBound << " at index " << upperBound -
arr2 << "\n";
        }
    }

    // std::equal_range는 이진 검색으로
    // value와 같은 원소의 범위를 찾는다.
    std::cout << "\n[equal_range]\n";
    // pair는 단순히 2가지의 데이터를
    // 담을 수 있는 타입이다.
    std::pair<int*, int*> range = std::equal_range(arr2, arr2 + 6,
5);
    if (range.first == arr2 + 3 && range.second == arr2 + 5)
    {
        std::cout << "arr2에서 5가 있는 곳은 arr2의 세 번째 원소부터 네
번째 원소까지다.\n";
    }

}
```

실습해보기

- 1. 아래의 문제를 풀어보자.
    - 1920번: 수 찾기
    - 10816번: 숫자 카드 2
    - 1654번: 랜선 자르기
    - 2805번: 나무 자르기
    - 2110번: 공유기 설치

연결 리스트

연결 리스트는 STL 상에서는 std::forward_list, std::list로 구현되어 있다. 선형 리스트와 다르게 임의 접근이 불가능하다.

- **읽기**

연결 리스트는 임의 접근이 불가능해 요소 하나하나를 탐색해야 하므로 $O(n)$의 시간이 걸린다. 하지만 처음과 끝이라면 구현에 따라 $O(1)$이 될 수 있다.

- **검색**

하나하나 원소를 비교해가야 하므로 $O(n)$의 시간이 걸린다. 선형 리스트와 다르게 이진 검색을 사용할 수 없다.

- **삽입**

원소를 어디에 삽입하냐에 따라 시간이 달라진다. 앞이나 뒤에 데이터를 추가할 경우 $O(1)$이지만, 중간이라면 해당 위치까지 검색해야 하기 때문에 $O(n)$이 걸린다.

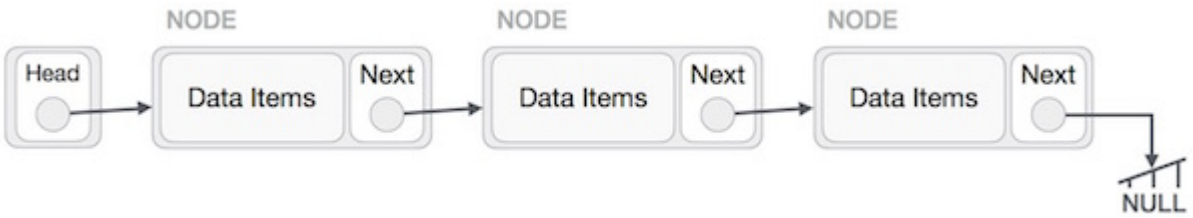
- **삭제**

삽입과 마찬가지로 원소를 어디에서 삭제하냐에 따라 시간이 달라진다. 앞이나 뒤의 데이터를 삭제할 경우 $O(1)$이지만, 중간이라면 $O(n)$이 걸린다.

종류

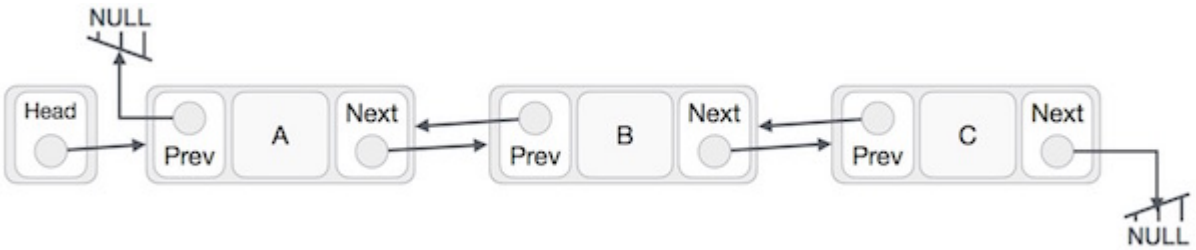
연결 리스트의 종류는 크게 세 가지다

- 단일 연결 리스트(Singly Linked List)
  - 다음 원소로만 이동할 수 있으며, std::forward_list가 여기에 해당한다.



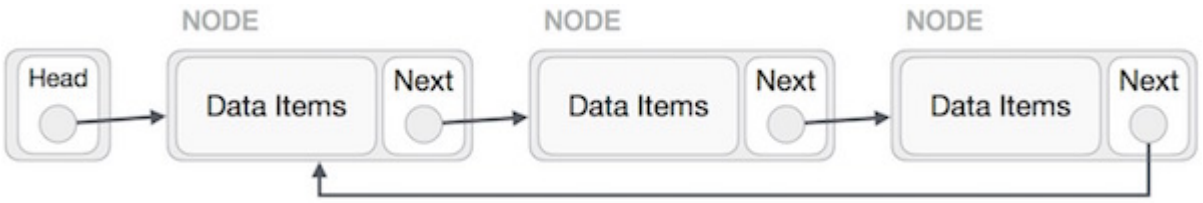
단일 연결 리스트

- 이중 연결 리스트(Doubly Linked List)
  - 이전 원소, 다음 원소 모두 이동할 수 있다.

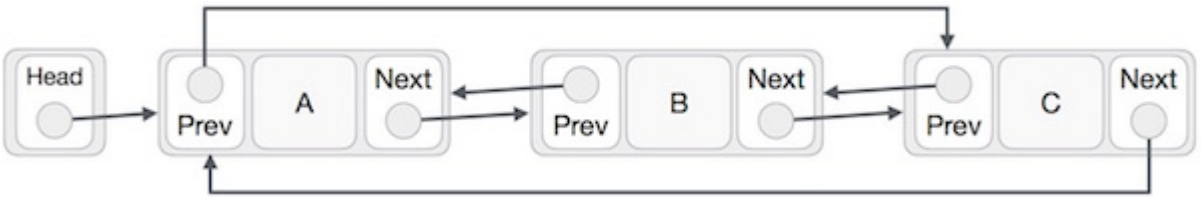


이중 연결 리스트

- 원형 연결 리스트(Circular Linked List)
  - 마지막 원소와 첫 번째 원소를 연결한 형태다.
  - std::list가 이중 원형 연결 리스트에 해당한다.



단일 원형 연결 리스트



이중 원형 연결 리스트

std::forward_list

```
#include <forward_list> // std::forward_list를 쓰기 위한 헤더
#include <iostream>

int main()
{
    std::forward_list<int> flist;

    // 삽입
    flist.push_front(1); // push_front() : 맨 앞에 삽입한다.
    // flist{ 1 }
    flist.insert_after(flist.begin(), 2); // insert_after(pos, value)
: pos 뒤에 value를 삽입한다.
    // flist{ 1, 2 }
    flist.push_front(3); // flist { 3, 1, 2 }

    // 반복자
    std::forward_list<int>::iterator iter;
    iter = flist.before_begin();
    // [ ]->[3]->[1]->[2]->[ ]
    //   ↑
    iter = flist.begin();
    // [ ]->[3]->[1]->[2]->[ ]
    //        ↑
    iter = flist.end();
    // [ ]->[3]->[1]->[2]->[ ]
```

```
    //                          ↑

    // 삭제
    flist.pop_front(); // pop_front() : 첫 번째 원소를 삭제한다.
    // flist{ 1, 2 };
    flist.erase_after(flist.begin()); // erase_after(pos) : pos 다음
원소를 삭제한다.
    // flist{ 1 }
    flist.clear(); // clear() : 컨테이너를 전부 비운다.
    // flist{ }

    // 크기
    if (flist.empty()) // empty() : 비었는지 확인한다.
    {
        std::cout << "flist는 비었음.\n";
    }
    // 주의! 다른 컨테이너와 다르게 size()는 없음

    // 아래처럼 초기화 가능
    std::forward_list<int> flist2 = { 1, 2, 3, 4, 5 };

    // 읽기
    std::cout << "flist2.front() : " << flist2.front() << "\n"; //
front() : 첫 번째 원소를 반환한다.

    // 비교 : 다른 컨테이너와 마찬가지로 == / != / > / >= / < / <= 지원
    flist = flist2;
    if (flist == flist2)
    {
        std::cout << "flist는 flist2와 같다.\n";
    }

    // C++17부터는 원소 타입을 적지 않아도 알아서 추론한다.
    std::forward_list flist3 = { 1, 2, 3, 4, 5 };

    return 0;
}
```

std::list

```
#include <list> // std::list를 쓰기 위한 헤더
#include <iostream>

int main()
{
    std::list<int> list;

    // 삽입
    list.push_front(1); // push_front() : 맨 앞에 삽입한다.
    // list{ 1 }
    list.push_back(2); // push_back() : 맨 뒤에 삽입한다.
    // list{ 1, 2 }
    list.insert(list.begin(), 9); // insert(pos, value) : pos 전에
value를 삽입한다.
    // list{ 9, 1, 2 }
    list.push_back(5); // list{ 9, 1, 2, 5 }

    // 반복자
    std::list<int>::iterator iter;
    iter = list.begin();
    // [9]<->[1]<->[2]<->[5]<->[ ]
    //  ↑
    iter = list.end();
    // [9]<->[1]<->[2]<->[5]<->[ ]
    //                         ↑
    std::list<int>::reverse_iterator riter;
    riter = list.rbegin();
    // [ ]<->[9]<->[1]<->[2]<->[5]
    //                         ↑
    riter = list.rend();
    // [ ]<->[9]<->[1]<->[2]<->[5]
    //  ↑

    // 삭제
    list.pop_front(); // pop_front() : 첫 번째 원소를 삭제한다.
    // list{ 1, 2, 5 };
    list.pop_back(); // pop_back() : 마지막 원소 삭제한다.
    // list{ 1, 2 };
    list.erase(list.begin()); // erase(pos) : pos를 삭제한다.
    list.clear(); // clear() : 컨테이너를 전부 비운다.
    // list{ }
```

```
    // 크기
    if (list.empty()) // empty() : 비었는지 확인한다.
    {
        std::cout << "list는 비었음.\n";
    }
    std::cout << "list.size() : " << list.size() << "\n";

    // 아래처럼 초기화 가능
    std::list<int> list2 = { 1, 2, 3, 4, 5 };

    // 읽기
    std::cout << "list2.front() : " << list2.front() << "\n"; //
front() : 첫 번째 원소를 반환한다.
    std::cout << "list2.back() : " << list2.back() << "\n"; // back()
: 마지막 원소를 반환한다.

    // 비교 : 다른 컨테이너와 마찬가지로 == / != / > / >= / < / <= 지원
    list = list2;
    if (list == list2)
    {
        std::cout << "flist는 flist2와 같다.\n";
    }

    // C++17부터는 원소 타입을 적지 않아도 알아서 추론한다.
    std::list list3 = { 1, 2, 3, 4, 5 };

    return 0;
}
```

std::forward_list 구현

먼저 단일 연결 리스트인 std::forward_list를 구현해보자.

```
class ForwardList
{
    struct Node
    {
        Node(int data = 0, Node* next = nullptr);
        Node(const Node&) = delete;
        Node& operator=(const Node&) = delete;
        ~Node();
```

```
        int     Data = 0;
        Node* Next = nullptr;
    };

public:
    class const_iterator
    {
    public:
        const_iterator(Node* p = nullptr);
        ~const_iterator();

        const int& operator*() const;
        const int* operator->() const;
        const_iterator& operator++();
        const_iterator      operator++(int);
        bool                operator==(const const_iterator& rhs)
const;
        bool                operator!=(const const_iterator& rhs)
const;
        bool                operator==(nullptr_t p) const;
        bool                operator!=(nullptr_t p) const;

    public:
        Node* _p = nullptr;
    };

    class iterator : public const_iterator
    {
    public:
        using const_iterator::const_iterator;

        int& operator*() const;
        int* operator->() const;
        iterator& operator++();
        iterator        operator++(int);
    };

    // 기본 생성자
    ForwardList();

    // count만큼의 요소를 갖고 있는 컨테이너를 만드는 생성자
    explicit ForwardList(size_t count);
```

```
// 복사 생성자.
ForwardList(const ForwardList& other);

// 할당 연산자
ForwardList& operator=(const ForwardList& rhs);

// 소멸자
~ForwardList();

// 첫 번째 요소를 반환한다.
int& front();
const int& front() const;

// 시작 전 요소를 가리키는 반복자를 반환한다.
iterator            before_begin();
const_iterator      before_begin() const;

// 시작 요소를 가리키는 반복자를 반환한다.
iterator            begin();
const_iterator      begin() const;

// 끝 다음 요소를 가리키는 반복자를 반환한다.
iterator            end();
const_iterator      end() const;

// pos 다음에 value를 삽입한다.
// 삽입된 요소를 가리키는 반복자를 반환한다.
iterator            insert_after(const_iterator pos, int value);

// pos 다음 요소를 삭제한다.
iterator            erase_after(const_iterator pos);

// 시작 요소에 value를 삽입한다.
void                push_front(int value);

// 시작 요소를 제거한다.
void                pop_front();

// 컨테이너가 비었는지 판단한다.
bool                empty() const;
```

```
    // 컨테이너를 비워버린다.
    void                clear();

    // value가 있는지 검사한다.
    bool                contains(int value) const;
private:
    Node* _beforeBegin = new Node();
};
```

std::list 구현

다음은 std::list를 구현해보자.

```
class List
{
public:
    struct Node
    {
        Node(int data = 0, Node* prev = nullptr, Node* next =
nullptr);
        Node(const Node&) = delete;
        Node& operator=(const Node&) = delete;
        ~Node() = default;

        int     Data = 0;
        Node* Next = nullptr;
        Node* Prev = nullptr;
    };

public:
    class const_iterator
    {
    public:
        const_iterator(Node* p = nullptr);
        ~const_iterator();

        const int& operator*() const;
        const int* operator->() const;

        const_iterator&     operator++();
        const_iterator      operator++(int);
        const_iterator&     operator--();
```

```
        const_iterator      operator--(int);

        bool                operator==(const const_iterator& rhs)
const;
        bool                operator!=(const const_iterator& rhs)
const;
        bool                operator==(nullptr_t p) const;
        bool                operator!=(nullptr_t p) const;

    public:
        Node* _p = nullptr;
    };

    class iterator : public const_iterator
    {
    public:
        using const_iterator::const_iterator;

        int& operator*() const;
        int* operator->() const;

        iterator& operator++();
        iterator        operator++(int);
        iterator& operator--();
        iterator        operator--(int);
    };

    // 기본 생성자
    List();

    // count만큼의 요소를 갖고 있는 컨테이너를 만드는 생성자
    explicit List(size_t count);

    // 복사 생성자.
    List(const List& other);

    // 할당 연산자
    List& operator=(const List& rhs);

    // 소멸자
    ~List();
```

```
    // 첫 번째 요소를 반환한다.
    int& front();
    const int& front() const;

    // 마지막 요소를 반환한다.
    int& back();
    const int& back() const;

    // 시작 요소를 가리키는 반복자를 반환한다.
    // 리스트가 비어있다면 end()와 같다.
    iterator            begin();
    const_iterator      begin() const;

    // 끝 다음 요소를 가리키는 반복자를 반환한다.
    iterator            end();
    const_iterator      end() const;

    // pos 이전에 value를 삽입한다.
    iterator            insert(iterator pos, int value);

    // pos 요소를 삭제한다.
    iterator            erase(iterator pos);

    // 시작에 value를 삽입한다.
    void            push_front(int value);

    // 끝에 value를 삽입한다.
    void            push_back(int value);

    // 시작 요소를 제거한다.
    void            pop_front();

    // 끝 요소를 제거한다.
    void            pop_back();

    // 컨테이너가 비었는지 판단한다.
    bool            empty() const;

    // 리스트 안에 있는 요소의 개수를 반환한다.
    size_t          size() const;

    // 컨테이너를 비워버린다.
```

```
    void            clear();

    // 해당 value가 있는지 체크한다.
    bool            contains(int value) const;
private:
    Node*    _end = new Node();
    size_t   _size = 0;
};
```

실습해보기

- 1. 아래의 문제를 풀어보자.
    - 1406번: 에디터 (acmicpc.net)
    - 5397번: 키로거 (acmicpc.net)

스택

스택(Stack)은 가장 마지막에 삽입된 데이터가 가장 먼저 나오는 **LIFO**(Last-In First-Out) 구조다. 스택의 끝을 **위**(Top), 스택의 시작을 **밑**(Bottom)이라 한다. 괄호가 올바른지 검사 / 후위 표기식 / DFS 등에 사용될 수 있다. STL에서는 std::stack으로 구현되어 있다.

std::stack

std::stack은 아래와 같이 사용한다.

```
#include <stack> // std::stack을 위한 헤더
#include <iostream>

int main()
{
    std::stack<int> s;

    // 삽입
    for (int i = 1; i <= 5; ++i)
    {
        s.push(i); // push() : 스택에 데이터를 삽입한다.
    }
    // s { 5, 4, 3, 2, 1 }
```

```
    // 컨테이너 어댑터는 반복자가 존재하지 않는다.
    //  => 순회 용도로 사용하지 않기 때문이다.

    // 삭제
    s.pop(); // pop() : 스택의 위에서 값을 꺼낸다.
    // s { 4, 3, 2, 1 }

    // 읽기
    std::cout << "stack.top() : " << s.top() << "\n";
    // top() : 가장 마지막으로 삽입된 원소를 가져온다.

    // 크기
    if (s.empty())
    {
        std::cout << "stack is empty\n";
    }
    std::cout << "stack.size() : " << s.size() << "\n";
    // size() : 현재 원소의 개수를 반환한다.
}
```

실습해보기

1. 아래의 문제를 풀어보자.
    a. 10828번: 스택 (acmicpc.net)
    b. 10773번: 제로
    c. 9012번: 괄호 (acmicpc.net)
    d. 4949번: 균형잡힌 세상

큐

큐(Queue)는 스택과 다르게 가장 처음에 들어간 데이터가 처음에 나오는 **FIFO**(First-In First-Out) 자료구조다. 삽입이 일어나는 쪽을 **뒤**(Rear), 삭제가 일어나는 쪽을 **앞**(Front)라고 한다. 프린트 큐 / CPU 스케줄링 / 데이터 버퍼 / BFS 등에 사용된다. STL에서는 std::queue, std::deque로 구현되어 있다.

std::queue

std::queue는 아래와 같이 사용한다.

```
#include <queue> // std::queue를 사용하기 위한 헤더
#include <iostream>

int main()
{
    std::queue<int> q;

    // 삽입
    for (int i = 1; i <= 5; ++i)
    {
        q.push(i);
    }
    // q { 1, 2, 3, 4, 5 }

    // 삭제
    q.pop();
    // q { 2, 3, 4, 5 }

    // 읽기
    std::cout << "q.front() : " << q.front() << "\n";
    std::cout << "q.back() : " << q.back() << "\n";

    // 크기
    if (q.empty())
    {
        std::cout << "q is empty\n";
    }
    std::cout << "q.size() : " << q.size() << "\n";
}
```

std::deque

데크는 Double-ended Queue로 양 끝에서 데이터 삽입 및 삭제가 가능한 자료구조다. std::deque의 경우 선형 리스트와 연결 리스트를 조합한 청크 리스트(Chunk List)라는 것으로 구현되어 있다. 각각의 특징을 모두 모아서 연산이 매우 효율적인지라 모든 컨테이너 어댑터의 기본 자료구조로 사용되고 있다. 사용 방법은 아래와 같다.

```
#include <deque> // std::deque를 사용하기 위한 헤더
#include <iostream>
```

```
int main()
{
    std::deque<int> dq;

    // 삽입
    dq.push_front(1); // dq{ 1 }
    dq.push_back(2); // dq{ 1, 2 }
    dq.insert(dq.begin() + 1, 3); // dq{ 1, 3, 2 }

    // 반복자
    std::deque<int>::iterator iter;
    iter = dq.begin();  // [*1][3][2][]
    iter = dq.end();    // [1][3][2][*]
    std::deque<int>::reverse_iterator riter;
    riter = dq.rbegin(); // [][1][3][*2]
    riter = dq.rend();   // [*][1][3][2]

    // 삭제
    dq.pop_front();          // dq{ 3, 2 }
    dq.pop_back();           // dq{ 3 }
    dq.erase(dq.begin());    // dq{ }

    // 아래처럼 초기화도 가능하다.
    std::deque<int> dq2 = { 1, 2, 3, 4, 5 };

    // 읽기
    std::cout << "dq2.front() : " << dq2.front() << "\n"; // 1
    std::cout << "dq2.back() : " << dq2.back() << "\n"; // 5
    std::cout << "dq2[2] : " << dq2[2] << "\n"; // 3

    // 크기
    std::cout << "dq2.size() : " << dq2.size() << "\n"; // 5

    // 비우기
    dq2.clear();
    if (dq2.empty())
    {
        std::cout << "dq2 is empty\n";
    }

    // C++17.부터는 원소의 타입 추론 가능
    std::deque dq3 = { 1, 2, 3 }; // deque<int>
```

```
}
```

실습해보기

1. 아래의 문제를 풀어보자
    a. 18258번: 큐 2
    b. 2164번: 카드2
    c. 11866번: 요세푸스 문제 0
    d. 10866번: 덱

재귀

재귀(recursion)는 함수가 자기 자신을 호출하는 것을 말한다. 재귀의 구조는 재귀를 중단시키는 **기저 조건**(base case)과 기저 조건으로 수렴하게 되는 **재귀 조건**(recursive case)으로 구성된다.

재귀호출을 사용하는 이유는 문제에 따라 전체를 한 번에 해결하기보다 같은 유형의 하위 작업으로 분할하여 작은 문제부터 해결하는 방법이 효율적*일 수 있기 때문이다. 다시 말해 복잡한 알고리즘을 단순하고 알기 쉽게 표현할 수 있다. 그뿐만 아니라 알고리즘 자체만으로는 얼마나 많은 단계를 깊이 들어가야 하는지 알 수 없을 때에도 사용할 수 있다.
*이를 분할 정복법(Divide & Conquer)이라고 한다.

예시를 살펴보자. 어떤 디렉토리가 갖고 있는 모든 파일 및 디렉토리를 지우는 함수를 구현한다고 해보자. 디렉토리에는 하위 디렉토리가 있을 수 있고, 그 하위 디렉토리에는 또 다른 하위 디렉토리가 있을 수 있으므로 같은 유형의 작업으로 분할할 수 있다.

```
void RemoveAll(directory):
  // 재귀 조건
  for childDirectory in childDirectories:
    RemoveAll(childDirectory)

  // 기저 조건
  모든 파일을 지운다;
```

재귀에는 한 가지 단점이 존재하는데, 함수 호출에 따른 **오버헤드**(overhead)*가 있다는 것이다. 재귀가 너무 깊어지면 그만큼 호출 스택의 공간을 더 많이 사용하게 되고, 끝내는 **스택 오버플로**(stack overflow)**가 발생할 수 있다.

*오버헤드 : 어떤 처리를 하기 위해 들어가는 간접적인 처리 시간 · 메모리 등을 말한다.
**스택 오버플로 : 스택이 가용할 수 있는 공간을 벗어나는 것을 말한다.

재귀를 연습해보기 위해 아래의 문제들을 풀어보자

- 10872번: 팩토리얼
- 10870번: 피보나치 수 5
- 10진수 입력 받아서 재귀로 2진수로 출력하기
- 17478번: 재귀함수가 뭐가요?
- 11729번: 하노이 탑 이동 순서
- 2447번: 별 찍기 - 10

그래프

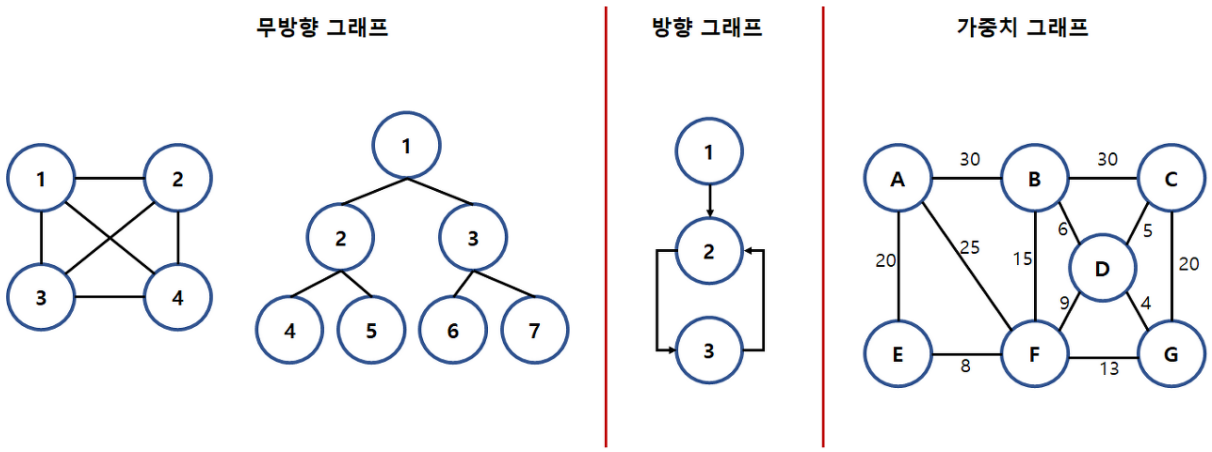
선형 자료구조로는 표현할 수 없는 문제가 있다. 회사의 조직도와 같은 계층을 표현한다던가 인스타그램 친구 관계도를 표현하는 것처럼 말이다. 이번 챕터부터는 이런 비선형적 문제를 다룰 수 있는 자료구조에 대해서 배우게 된다.

그래프란?

그래프(graph)는 관계에 특화된 자료구조로 **정점**(Vertex)과 **간선**(Edge)으로 구성되어 있다. 각 정점은 고유하게 식별되는 객체이고, 간선은 정점 간의 관계를 표현한다.

종류

그래프의 종류에는 간선이 방향성을 가지는 지에 따라 **방향 그래프**(Directed Graph)와 **무방향 그래프**(Undirected Graph)로 구분할 수 있으며, 간선이 단순 연결 이상의 정보(이를 가중치라 한다.)를 가지는 **가중치 그래프**(Weighted Graph)*도 있다.
*네트워크(Network)라고도 한다.



그래프의 종류

용어

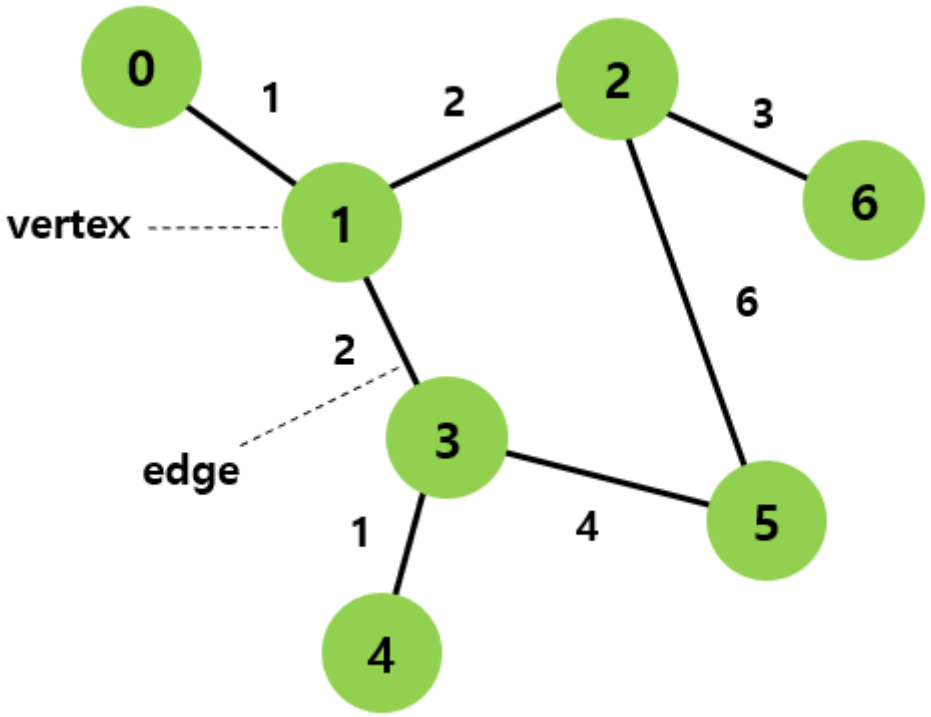
그래프와 관련된 여러가지 용어가 있다. 먼저 간선으로 연결된 정점끼리는 **인접**(Adjacent)하다고 하며, 해당 간선은 연결된 정점에 **부속**(Incident)되어 있다고 한다. 정점에 부속되어 있는 간선의 수를 **차수**(Degree)라고 하며 정점으로 들어오는 방향인지, 나가는 방향인지 구분해 **진입 차수**(In-degree), **진출 차수**(Out-degree)로 말할 수 있다.

한 정점에서 다른 정점까지 간선으로 연결된 정점을 순서대로 나열한 리스트를 **경로**(Path)라 한다. 경로를 구성하는 간선 수를 **경로 길이**(Path length)라 하며, 모두 다른 정점으로 구성된 경로를 **단순 경로**(Simple Path)라 한다. 단순 경로 중 경로의 시작 정점과 마지막 정점이 같은 경로를 **사이클**(Cycle)이라 한다. 또, 경로가 있으면 정점끼리 **연결***(Connected) 되었다고 한다.

*모든 정점이 연결되어 있다면 연결 그래프(Connected Graph), 아니면 단절 그래프(Disconnected Graph)라 한다.

표현

그래프를 표현하는 방법에는 **인접 행렬**(Adjacency Matrix)과 **인접 리스트**(Adjacency List)가 있다. 아래의 그래프를 각각 인접 행렬과 인접 리스트로 나타내본다.



인접 행렬로 표현하기

정점이 N개 있을 경우 N x N 크기의 2차원 배열로 그래프를 표현할 수 있다. 배열의 원소는 간선의 가중치를 나타내게 된다. 가령 edges[i][j] 는 정점 i 에서 j로 가는 간선을 나타낸 것이다. 2차원 배열을 사용하기 때문에 그래프에 간선이 적다면* 그만큼 메모리를 낭비하게 된다.

*이를 희소 그래프(Sparse Graph)라 한다. 반대는 밀집 그래프(Dense Graph)다.

```
// 정점이 7개이므로 7 x 7의 2차원 배열을 정의한다.
int graph[7][7];

// 무방향 그래프이므로 서로 연결해준다.
graph[0][1] = 1; // 0 -> 1의 방향과 1의 가중치를 가진 엣지
graph[1][0] = 1; // 1 -> 0의 방향과 1의 가중치를 가진 엣지

graph[1][2] = 2; // 1 -> 2의 방향과 2의 가중치를 가진 엣지
graph[2][1] = 2; // 2 -> 1의 방향과 2의 가중치를 가진 엣지
graph[1][3] = 2; // 1 -> 3의 방향과 2의 가중치를 가진 엣지
graph[3][1] = 2; // 3 -> 1의 방향과 2의 가중치를 가진 엣지

// 이하 설명 생략
graph[2][5] = 6;
graph[5][2] = 6;
graph[2][6] = 3;
```

```
graph[6][2] = 3;

graph[3][4] = 1;
graph[4][3] = 1;
graph[3][5] = 4;
graph[5][3] = 4;
```

인접 리스트로 표현하기

인접 리스트는 각 정점에 연결된 정점의 ID만 저장하는 방식으로 그래프를 표현하는 것이다.

```
#include <vector>
#include <utility>

using namespace std;

// 정점이 7개이므로 크기가 7인 배열을 만든다.
// pair의 first에는 정점의 ID를 저장하고,
// pair의 second에는 가중치를 저장한다.
// 각 정점에 대한 진출 간선을 저장한다고 생각하면 된다.
vector<pair<int, int>> graph[7];

graph[0].emplace_back(1, 1);

graph[1].emplace_back(0, 1);
graph[1].emplace_back(2, 2);
graph[1].emplace_back(3, 2);

graph[2].emplace_back(1, 2);
graph[2].emplace_back(5, 6);
graph[2].emplace_back(6, 3);

grpah[3].emplace_back(1, 2);
graph[3].emplace_back(4, 1);
graph[3].emplace_back(5, 4);

graph[4].emplace_back(3, 1);

graph[5].emplace_back(2, 6);
graph[5].emplace_back(3, 4);

graph[6].emplace_back(2, 3);
```

순회

한 정점에서 시작해 그래프에 있는 모든 정점을 한 번씩 방문하는 것을 **그래프 순회**(Graph Traversal) 또는 **탐색**(Search)이라 한다. 이러한 방법에는 **깊이 우선 탐색**(Depth First Search)과 **너비 우선 탐색**(Breadth First Search)이 있다.

깊이 우선 탐색은 시작 정점의 한 방향으로 갈 수 있는 경로가 있는 곳까지 깊이 탐색해 가다 더 이상 갈 곳이 없으면 가장 마지막에 만났던 갈림길 간선이 있는 정점으로 되돌아와 다른 방향의 간선으로 탐색을 계속하는 방법으로 스택을 사용한다.

너비 우선 탐색은 시작 정점에 인접한 정점을 모두 차례로 방문하고 나서 방문했던 정점을 시작으로 다시 인접한 정점을 차례로 방문하는 방식으로 큐를 사용한다.

실습해보기

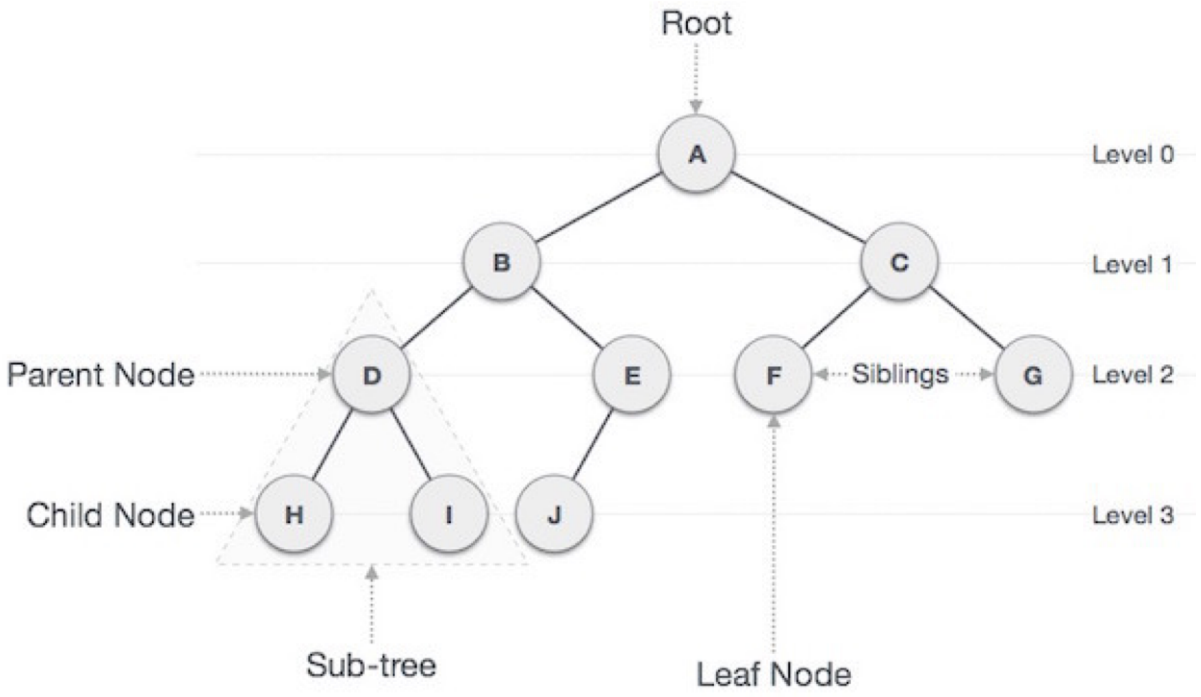
- 1. 아래의 문제를 풀어보자.
    - 1260번: DFS와 BFS
    - 2606번: 바이러스
    - 2667번: 단지번호붙이기
    - 1012번: 유기농 배추
    - 2178번: 미로 탐색
    - 7576번: 토마토
    - 7569번: 토마토
    - 1697번: 숨바꼭질
    - 7562번: 나이트의 이동

트리

트리(Tree)는 그래프의 일종으로 계층형 자료구조(Hierarchical Data Structure)다. 트리는 데이터가 저장된 **노드**(Node)와 노드 간 관계를 표현하는 **간선**(Edge)으로 구성된다.

용어

트리에는 여러 가지 용어가 있다. 아래의 그림을 참고하자.



이 외의 용어는 아래와 같다.

- **조상**(Ancestor) 노드 : 한 노드에서 간선을 따라 루트 노드까지 이르는 경로에 있는 모든 노드
- **자손**(Descendant) 노드 : 조상 노드의 반대
- **차수**(Degree) : 한 노드가 가지는 서브 트리의 수. 노드의 차수 중 최댓값을 트리의 차수라 한다.
- **내부**(Internal) 노드 : 단말 노드를 제외한 나머지 노드
- **포레스트**(Forest) : 트리의 집합

순회

트리에 대한 순회는 **전위 순회**(pre-order traversal) / **중위 순회**(in-order traversal) / **후위 순회**(post-order traversal) / **레벨 순회**(level-order traversal)가 있다. 위의 이미지를 토대로 순회 순서를 나타내면 아래와 같다.

- 전위 순회 : A - B - D - H - I - E - J - C - F - G
- 중위 순회 : H - D - I - B - J - E - A - F - C - G
- 후위 순회 : H - I - D - J - E - B - F - G - C - A
- 레벨 순회 : A - B - C - D - E - F - G - H - I - J

이진 검색 트리

이진 검색 트리(Binary Search Tree)는 한 노드를 기준으로 왼쪽 서브 트리는 모두 그 노드보다 작은 값을 가지고 있으며, 오른쪽 서브 트리는 모두 그 노드보다 큰 값을 가지고 있는 **이진 트리***(Binary tree)이다. 수식으로 표현하면 **왼쪽 자식 노드의 값 <= 부모 노드의 값 <= 오른쪽 자식 노드의 값**이 된다. STL에 구현된 트리 컨테이너는 모두 이진 검색 트리다. 이진 검색 트리를 중위 순회하면 정렬된 데이터를 얻을 수 있다.

*이진 트리(binary tree) : 트리의 차수가 2 이하인 트리

트리의 균형

이진 탐색 트리는 데이터의 삽입 순서에 따라 형태가 달라지는데, 이에 따라 **트리의 균형도**가 결정된다. 트리의 균형도는 왼쪽 서브 트리의 높이와 오른쪽 서브 트리의 높이의 차를 말하는데, 균형도의 절댓값이 2이상이면 트리가 편향되었다고 표현한다. 트리의 균형도는 트리의 연산에 영향을 미친다. 트리가 균형잡혀 있다면 모든 연산이 $O(logN)$이지만, 편향되어 있다면 $O(N)$에 가까워진다.

그래서 어떻게 하면 트리의 균형을 유지할 수 있을까를 고민했고, 이에 따라 **자가 균형 이진 검색 트리**(Self-balancing Binary Search Tree)가 생겨나게 되었다. 여기에는 AVL 트리와 레드 블랙 트리가 존재한다.

구현

이진 검색 트리를 구현해 집합 자료구조를 구현해보자. 집합은 유일한 데이터만을 가지는 자료구조이다.

```
class Set
{
    struct Node
    {
        Node(int data = 0, Node* parent = nullptr, Node* left =
nullptr, Node* right = nullptr);
        Node(const Node&) = delete;
        Node& operator=(const Node&) = delete;
        ~Node();

        bool    IsLeaf() const;

        int     Data = 0;
        Node* Parent = nullptr;
        Node* Left = nullptr;
        Node* Right = nullptr;
```

```
    };

public:
    // 트리의 높이를 구한다.
    int         height() const; // 큐를 이용한다.
    int         height2() const; // 재귀를 이용한다.

    // 트리가 비었는지 확인한다.
    bool        empty() const;

    // 트리의 크기를 반환한다.
    size_t      size() const;

    // 트리를 비운다.
    void        clear();

    // 트리에 값을 삽입한다.
    bool        insert(int value);

    // 트리에서 값을 삭제한다.
    size_t      erase(int value);

    // 트리에서 값을 찾는다.
    Node*       find(int value) const;

    // 순회
    void        traverseByPreorder() const;
    void        traverseByInorder() const;
    void        traverseByPostorder() const;
    void        traverseByLevelorder() const;

private:
    Node*   _head = new Node();
    size_t  _size = 0;
};
```

STL

STL에서는 std::set / std::map / std::multiset / std::multimap 으로 구현이 되어 있다. 이들을 STL에서는 **연관 컨테이너**(Associative Container)라고 하는데, 트리의 노드가 이진 검색 트리를

구성하는데 사용하는 키(Key)와 이 키에 연관되는 값(Value)의 형태로 구조가 만들어져 있기 때문이다. multi가 붙은 자료구조는 데이터의 중복을 허용한다는 의미이고, 사용법은 같기 때문에 여기서는 set과 map만 예제 코드를 작성한다.

std::set

집합이라는 이름 그대로 유일한 데이터를 저장한다.

```
#include <set> // set을 쓰기 위한 헤더
#include <iostream>

int main()
{
    std::set<int> s;

    // 삽입
    s.insert(1); // s{ 1 }
    s.insert(2); // s{ 1, 2 }
    s.insert(3); // s{ 1, 2, 3 }

    // 중복된 데이터는 삽입할 수 없다.
    // insert()의 반환은 pair 타입이며
    // 두 번째 데이터에 삽입 여부가 담긴다.
    if (false == s.insert(1).second)
    {
        std::cout << "이미 1을 가지고 있음.\n";
    }

    // 반복자
    // 다른 컨테이너와 비슷하다.
    std::set<int>::iterator iter;
    iter = begin(); // [*1][2][3][]
    iter = end();   // [1][2][3][*]
    std::set<int>::reverse_iterator riter;
    riter = rbegin(); // [][1][2][*3]
    riter = rend(); // [*][1][2][3]

    // 기타 연산
    // count() : 데이터가 몇 개인지 센다.
    size_t c = s.count(1); // 1을 반환한다.
    // find() : 데이터가 있는지 확인한다.
    if (s.end() == s.find(4)) // 없으면 end()다.
```

```
    {
        std::cout << "4는 없다.\n";
    }

    // 삭제
    s.erase(2); // s{ 1, 3 }
    s.clear(); // s{}

    // 크기
    if (0 == s.size())
    {
        std::cout << "s의 size는 0임\n";
    }
    if (s.empty())
    {
        std::cout << "s는 비었음\n";
    }

    // 비교가 가능하다.
    std::set<int> s2(s);
    if (s == s2)
    {
        std::cout << "둘이 같음\n";
    }

    // C++17부터는 아래와 같이 초기화가 가능하다.
    std::set s3 = { 1, 2, 3, 4 };
}
```

std::map

map의 사용도 크게 다르지 않다.

```
#include <map> // map을 쓰기 위한 헤더
#include <iostream>
#include <string>

int main()
{
    // 첫 번째 템플릿 파라미터는 키의 타입을
    // 두 번째 템플릿 파라미터는 값의 타입을 지정한다.
```

```
    std::map<std::string, int> ageMap;

    // 삽입
    // 첫 번째 원소는 트리에 사용되는 키고,
    // 두 번째 원소는 키에 대응되는 값이다.
    ageMap.insert({"홍길동", 23});
    ageMap.insert({"김철수", 17});

    // 원소 접근
    // [] 연산자로 ageMap에 있는 원소를 접근할 수 있다.
    std::cout << ageMap["홍길동"] << "\n"; // 23
    std::cout << ageMap["김철수"] << "\n"; // 17
    ageMap["홍길동"] = 25; // 이제 홍길동과 대응되는 데이터는 25다.
    ageMap["이영희"] = 19; // 새로운 데이터를 삽입할 수도 있다.

    // 반복자도 제공된다. 코드는 생략한다.

    // 기타 연산
    // set에서 제공하는 연산을 갖고 있다.
    size_t c = ageMap.count("홍길동"); // 1
    if (ageMap.end() != ageMap.find("이영희"))
    {
        std::cout << "영희와 관련된 데이터가 존재한다.\n";
    }

    // 삭제
    ageMap.erase("홍길동");
    ageMap.clear();

    // 크기
    ageMap.size(); // 0
    ageMap.empty(); // true

    // C++17부터는 아래처럼 초기화 할 수 있다.
    // 아래의 map의 타입은 map<string, string>이다.
    std::map tempMap = { std::pair{"Hello", "World"} };
}
```

실습해보기

- 1. 아래의 문제를 풀어보자.

- 11725번: 트리의 부모 찾기
- 1991번: 트리 순회
- 5639번: 이진 검색 트리

힙

힙(heap)은 완전 이진 트리에 있는 노드 중에서 키값이 가장 큰 노드나 키값이 가장 작은 노드를 찾기 위해 만든 자료구조다. 키값이 가장 큰 노드를 찾기 위한 힙을 **최대 힙**(max heap), 가장 작은 노드를 찾기 위한 힙을 **최소 힙**(min heap)이라고 한다. **우선순위 큐**(priority queue)라고도 한다. STL에서는 std::priority_queue로 구현이 되어 있다.

힙의 불변성

힙은 최대 / 최소 원소에 즉각적으로 접근이 가능해야 한다. 따라서 최대 / 최소 원소가 항상 고정된 위치에 있어야 한다. 그래서 **최대 / 최소 원소는 항상 트리의 루트에 존재**한다. 또, **부모 노드가 두 자식 노드보다 항상 크거나 작아야 한다.**

연산

- 검색 및 읽기
  - 최대 / 최소 원소에 대해서만 가능하며 $O(1)$이다.
- 삽입
  - 완전 이진 트리이기 때문에 $O(logN)$이다.
- 삭제
  - 최대 / 최소 원소에 대해서만 가능하며 $O(logN)$이다.

직접 구현해보자.

```
class Heap
{
public:
    Heap() = default;
    ~Heap() = default;

    // 힙의 가장 큰 원소를 반환한다.
    const int& top() const;
```

```
    // 힙이 비었는지 체크한다.
    bool        empty() const;

    // 힙의 크기를 반환한다.
    size_t      size() const;

    // 힙에 값을 삽입한다.
    void        push(int value);

    // 힙에서 값을 제거한다.
    void        pop();
private:
    vector<int>     _container;
};
```

실습해보기

- 1. 아래의 문제를 풀어보자.
    - 11279번: 최대 힙
    - 1927번: 최소 힙
    - 11286번: 절댓값 힙

해시 테이블

특정 값을 찾고자 할 때 배열을 사용하면

- 해싱(hashing) : 입력의 크기에 상관 없이 일정 크기의 값으로 변환하는 것. 이에 사용되는 함수를 해시 함수(hash function)라 한다.
    - 균일성 : 충돌이 적은 것을 의미한다.
        - 충돌(collision) : 서로 다른 값이 같은 해시 값을 생성한 것
    - 효율성 : 계산하기 쉬워야 한다.
    - 결정적 : 같은 입력에는 같은 값을 내놓아야 한다.
    - Ex. 디지털 서명, 공인 인증서 등등등등

- 해시 테이블(hash table) : 해싱을 활용해 데이터를 저장하는 검색을 위한 자료구조. 연관 배열(associative array)이라고도 한다. 해시 값을 테이블의 인덱스로 활용한다.
- 충돌 처리
    - 선형 개방 주소법 or 선형 조사법
        - 충돌이 나면 걍 빈 곳을 찾아서 넣는다.
    - 체이닝
        - 충돌이 나면 리스트로 여러 값을 저장하게 한다.

- 백트래킹
    - 상태공간트리
        - 찾는 해를 포함하는 트리
            - 즉, 해가 존재한다면 그것은 반드시 이 트리의 어떤 한 노드에 해당함. 따라서 이 트리를 체계적으로 탐색하면 해를 구할 수 있음
        - 상태공간 트리의 모든 노드를 탐색해야 하는 것은 아니다.
        - 루트에서 출발하여 체계적으로 모든 노드를 방문하는 절차를 기술한다.
    - 상태공간 트리를 DFS로 탐색하여 해를 찾는 알고리즘
- 멱집합

해시 테이블
- 연관 배열
- 키와 값의 쌍으로 이뤄진 리스트
- 해싱(hashing) : 문자를 가져와 숫자로 변환하는 과정.
- 해시 함수(hash function) : 글자를 특정 숫자로 변환하는 함수
    - 주어진 입력에 반환하는 결과가 일관되지 않으면 그 해시 함수는 유효한 게 아님
    - 사용 가능한 모든 셀에 고르게 데이터를 분산시키는 게 좋은 해시 함수
- 충돌
    - 이미 채워진 셀에 데이터를 추가하는 것
    - 해결법
        - 분리 연결법
            - 충돌이 발생했을 때, 하나의 값 대신 여러 값을 넣는 것
- 효율성에 영향을 미치는 요인 → 많은 메모리를 낭비하지 않으면서 균형을 유지하며 충돌을 피한다.
    - 얼마나 많은 데이터를 저장하는가?
    - 얼마나 많은 셀을 사용할 수 있는가? → 부하율
    - 어떤 해시 함수를 사용하는가?

- 집합에 사용
    - 중복 값을 허용하지 않는 구조
- 순서를 유지하지 못한다.

정렬

버블정렬

선택정렬

삽입정렬

-

- 분할 : 해결하고자 하는 문제를 작은 크기의 동일한 문제들로 분할
    - 정복 : 각각의 작은 문제를 순환적으로 해결
    - 합병 : 작은 문제의 해를 합하여 원래 문제의 대한 해를 구함
    - Merge Sort
        - 절차
            - 데이터가 저장된 배열을 절반으로 나눔
            - 각각을 순환적으로 정렬
            - 정렬된 두 개의 배열을 합쳐 전체를 정렬
        - 시간복잡도 : O(nlogn) / 공간복잡도 : O(n)

퀵 정렬

- 매우 빠른 정렬 방법 중 하나로 평균적인 경우에서 효율적이다.
    - 최악의 경우 : O(N^2)
    - 평균적인 경우 : O(NlogN)

TO DO : 퀵 정렬 구현 및 퀵 실렉트 구현

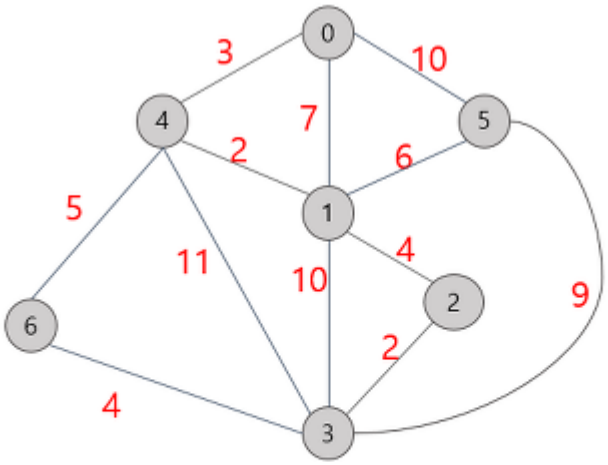
최단 경로

최단 경로(Shortest Path)는 가중치 그래프에서 두 정점을 연결하는 경로 중 가중치의 합이 최소인 경로를 말한다. 최단 경로를 구하기 위해 간선이 없다면 무한대 값으로 저장하며, 가중치가

음수여선 안된다. 최단 경로를 구하는 알고리즘에서는 다익스트라 알고리즘과 플로이드 알고리즘이 있다.

다익스트라 알고리즘

다익스트라 알고리즘(dijkstra algorithm)은 **하나의 시작 정점에서 다른 정점까지의 최단 경로**를 구하는 알고리즘으로 무방향 그래프, 방향 그래프 모두 적용 가능하다. 기본 원리는 시작 정점에서 가장 가까운 정점을 선택하여 추가하면서 추가된 모든 정점에 대한 최단 경로를 구해가는 과정이다. 아래의 그래프를 이용해 구현해보자.



출처 : [<Dijkstra의 최단 경로 알고리즘> 기본개념과 알고리즘]

코드는 아래와 같다.

```
#include <stdio.h>
#include <iostream>
#include <vector>
#include <algorithm>
#include <queue>

#define INF 987654321

using namespace std;

int graph[7][7] = {
```

```
    {  0,  7, INF, INF,  3, 10, INF },
    {  7,  0,  4, 10,  2,  6, INF },
    { INF,  4,  0,  2, INF, INF, INF },
    { INF, 10,  2,  0, 11,  9,  4 },
    {  3,  2, INF, 11,  0, INF,  5 },
    { 10,  6, INF,  9, INF,  0, INF },
    { INF, INF, INF,  4,  5, INF,  0 }
};

// start : 시작 정점
std::vector<int> GetShortestPath(int start)
{
    // distance 배열 만들기
    std::vector<int> dist(7);
    for (int i = 0; i < 7; ++i)
        dist[i] = graph[start][i];

    // 방문한 정점 집합 만들기
    std::vector<bool> isVisited(7);
    isVisited[start] = true;

    // 모든 정점에 대한 최단 경로를 찾을 때까지
    for (int i = 0; i < 7 - 1; ++i)
    {
        // 방문하지 않은 정점 중 최솟값을 가지는 정점을 구함
        size_t next = 0;
        size_t minDist = INF;

        for (size_t node = 0; node < 7; ++node)
        {
            if (isVisited[node])
                continue;
```

```
        if (minDist > dist[node])
        {
            next = node;
            minDist = dist[node];
        }
    }

    // 방문 표시 => 방문한 정점은 최소 거리임을 보장함.
    isVisited[next] = true;

    // 경로 업데이트
    for (size_t node = 0; node < 7; ++node)
    {
        dist[node] = std::min(dist[node], dist[next] + graph[next][node]);
    }
  }

  return dist;
}

std::vector<int> GetShortestPath2(int start)
{
  // distance 배열 만들기
  // 발견되지 않은 정점은 모두 INF다.
  vector<int> dist(7, INF);
  // 시작 정점은 거리가 0이다.
  dist[start] = 0;

  // 우선순위 큐를 이용한다.
  priority_queue<pair<int, int>> pq;
  // 시작 정점을 넣어준다.
```

```
    pq.emplace(0, start);

    // dist의 모든 값을 찾을 때까지
    while (false == pq.empty())
    {
        // dist 중에서 가장 최소값을 가지는 정점을 뽑아낸다.
        int w = -pq.top().first;
        int v = pq.top().second;
        pq.pop();

        // 과거에 발견한 최단 경로와 현재 최단 경로를 비교한다.
        // 과거에 발견한 최단 경로가 더 크다면 계산할 필요가 없다.
        if (dist[v] < w)
        {
            continue;
        }

        // 해당 정점을 이용했을 때, 최단 거리를 찾을 수 있는지 체크한다.
        for (int node = 0; node < 7; ++node)
        {
            if (dist[node] > w + graph[v][node])
            {
                dist[node] = w + graph[v][node];

                pq.emplace(-dist[node], node);
            }
        }
    }

    return dist;
}
```

```
std::vector<int> GetShortestPathWithPath(int start, vector<int>& outPath)
{
    vector<int> dist(7, INF);
    dist[start] = 0;

    // path[i]는 i 정점으로 가기 위해 거쳐야 하는 정점이다.
    vector<int> path(7, INF);
    path[start] = start;

    priority_queue<pair<int, int>> pq;
    pq.emplace(0, start);

    while (false == pq.empty())
    {
        int w = -pq.top().first;
        int v = pq.top().second;
        pq.pop();

        if (dist[v] < w)
        {
            continue;
        }

        for (int node = 0; node < 7; ++node)
        {
            if (dist[node] > w + graph[v][node])
            {
                dist[node] = w + graph[v][node];

                // 해당 node로 가기 위해서는 v를 거쳐야 한다.
                path[node] = v;
```

```
            pq.emplace(-dist[node], node);
        }
    }
  }

  outPath = std::move(path);

  return dist;
}

void PrintPath(int start, int end)
{
  vector<int> path;
  GetShortestPathWithPath(start, path);

  int curr = end;
  while (curr != start)
  {
    printf("%d <- ", curr);
    curr = path[curr];
  }
  printf("%d", curr);

  // 혹은 스택에 넣어서 순서대로 출력할 수도 있다.
}

int main()
{
  auto vec1 = GetShortestPath(0);
  auto vec2 = GetShortestPath(2);

  std::cout << "0 : [";
```

```
    for (int i : vec1)
    {
        std::cout << i << ' ';
    }
    std::cout << "]₩n";

    std::cout << "2 : [";
    for (int i : vec2)
    {
        std::cout << i << ' ';
    }
    std::cout << "]₩n";

    auto vec3 = GetShortestPath2(0);
    std::cout << "0 : [";
    for (int i : vec3)
    {
        std::cout << i << ' ';
    }
    std::cout << "]₩n";

    PrintPath(0, 2);
}
```

실습해보기

- 1. 아래의 문제를 풀어보자.
    - 1753번: 최단경로
    - 1504번: 특정한 최단 경로

A* 알고리즘

A* 알고리즘은 다익스트라 알고리즘을 확장한 최단 경로 알고리즘이다. 가중치를 계산할 때, 허용적 휴리스틱(admissible heuristic)*을 사용하게 되며, 휴리스틱이 0이라면 다익스트라 알고리즘과 정확히 일치한다. 휴리스틱으로는 맨하탄 거리, 유클리드 거리, 코사인 유사도 등이 쓰인다.

*휴리스틱(heuristic) :

```
#include <iostream>
#include <queue>
#include <vector>

#define START_X 2
#define START_Y 4
#define END_X   7
#define END_Y   4

#define INF 1e9

using namespace std;

int map[10][10] = {
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 2, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 2, 0, 0, 0, 0, 0 },
    { 0, 0, 1, 0, 2, 0, 0, 3, 0, 0 },
    { 0, 0, 0, 0, 2, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 2, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 2, 0, 0, 0, 0, 0 },
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
```

```
    { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }
};

struct Pos
{
    int X;
    int Y;

    bool operator<(const Pos& other) const { return (X < other.X) && (Y < other.Y); }
    bool operator==(const Pos& other) const { return X == other.X && Y == other.Y; }
    bool operator!=(const Pos& other) const { return !(*this == other); }
};

void PrintMap()
{
    for (int r = 0; r < 10; ++r)
    {
        for (int c = 0; c < 10; ++c)
        {
            if (map[r][c] == 0)
                cout << " ";
            else if (map[r][c] == 1)
                cout << "S";
            else if (map[r][c] == 2)
                cout << "B";
            else if (map[r][c] == 3)
                cout << "E";
            else if (map[r][c] == 4)
                cout << "P";
        }
        cout << endl;
    }
```

```
}

int h(Pos a, Pos b)
{
    return abs(a.X - b.X) + abs(a.Y - b.Y);
}

void astar(Pos start, Pos end)
{
    vector<vector<int>> f;
    for (int i = 0; i < 10; ++i)
        f.emplace_back(10, INF);
    f[start.Y][start.X] = 0;

    Pos path[10][10];
    path[start.Y][start.X] = start;

    priority_queue<pair<int, Pos>> pq;
    pq.emplace(0, start);

    int dy[] = { -1, -1, 0, 1, 1, 1, 0, -1 };
    int dx[] = { 0, 1, 1, 1, 0, -1, -1, -1 };
    int dg[] = { 10, 14, 10, 14, 10, 14, 10, 14 };

    while (false == pq.empty())
    {
        int w = -pq.top().first;
        Pos pos = pq.top().second;
        pq.pop();

        if (f[pos.Y][pos.X] < w)
        {
```

```
        continue;
    }

    if (pos == end)
    {
        break;
    }

    for (int i = 0; i < 8; ++i)
    {
        int ny = dy[i] + pos.Y;
        int nx = dx[i] + pos.X;

        if (ny < 0 || ny >= 10 || nx < 0 || nx >= 10)
        {
            continue;
        }

        if (!(map[ny][nx] == 0 || map[ny][nx] == 3))
        {
            continue;
        }

        int nf = dg[i] + h({ nx, ny }, end);

        if (f[ny][nx] > nf)
        {
            f[ny][nx] = nf;

            path[ny][nx] = pos;

            pq.emplace(-f[ny][nx], Pos{ nx, ny });
```

```
            }
        }
    }

    Pos curr = end;
    while (curr != start)
    {
        map[curr.Y][curr.X] = 4;

        curr = path[curr.Y][curr.X];
    }
}

int main()
{
    PrintMap();

    puts("");

    astar({ START_X, START_Y }, { END_X, END_Y });

    PrintMap();

}
```

플로이드 알고리즘

플로이드(floyd) 알고리즘*은 모든 정점 사이의 최단 경로를 구하는 알고리즘이다. 기본 원리는 아래와 같다.

*이렇게 구한 경로를 k-최단 경로라고도 한다.

- $A^k[v][w]$는 0부터 k까지의 정점만을 이용한 정점 v에서 정점 w까지의 최단 경로를 의미

- 정점 0부터 정점 k-1까지의 정점을 고려한 최단 거리 A^{k-1}을 구한 상태에서 다음 정점 k를 고려할 때, $A^{k-1}[v][w]$와 $A^{k-1}[v][k] + A^{k-1}[k][w]$중에서 작은 값에 따라 최단 경로 수정
- A^{-1}부터 A^{n-1}까지 순서대로 경로에 정점을 늘려가면서 최단 경로를 구하면 n개의 모든 정점 사이의 최단 경로를 구하게 된다.
- A^{-1}은 초기 상태로서 가중치 인접 행렬이 되고, $A^{n-1}[v][w]$는 정점 0부터 정점 n-1까지의 모든 정점을 이용한 최단 경로가 된다.

```
#include <stdio.h>
#include <iostream>
#include <vector>
#include <algorithm>

#define INF 987654321

using namespace std;

int graph[7][7] = {
   {  0,   7, INF, INF,   3,  10, INF },
   {  7,   0,   4,  10,   2,   6, INF },
   { INF,   4,   0,   2, INF, INF, INF },
   { INF,  10,   2,   0,  11,   9,   4 },
   {  3,   2, INF,  11,   0, INF,   5 },
   {  10,   6, INF,   9, INF,   0, INF },
   { INF, INF, INF,   4,   5, INF,   0 }
};

vector<vector<int>> GetShortestPath()
{
   // distance 인접 행렬 만들기
   // distance[i][j]는 i 정점에서 j 정점까지의 최단 경로다.
   vector<vector<int>> dist;
   for (int i = 0; i < 7; ++i)
```

```
        dist.emplace_back(graph[i], graph[i] + 7);

    // 모든 정점의 최단 경로를 구할 때까지 반복한다.
    for (int via = 0; via < 7; ++via) // 경유 정점
    {
        for (int from = 0; from < 7; ++from) // 시작 정점
        {
            for (int to = 0; to < 7; ++to) // 도착 정점
            {
                dist[from][to] = min(dist[from][to], dist[from][via] + dist[via][to]);
            }
        }
    }

    return dist;
}

int main()
{
    auto dist = GetShortestPath();

    for (const auto& d : dist)
    {
        cout << "[ ";
        for (const auto& elem : d)
        {
            cout << elem << ' ';
        }
        cout << "]₩n";
    }
}
```

실습해보기

- 1. 아래의 문제를 풀어보자
    - 11404번: 플로이드
    - 1956번: 운동

신장 트리

신장 트리(spanning tree)는 정점이 n개인 그래프에서 최소 개수의 간선으로 모든 정점을 연결한 트리 형태의 부분 그래프를 의미한다. 신장 트리는 도로를 최소화 해 모든 도시를 연결해야 하는 도로망 건설이나 네트워크 선을 최소로 사용해 시스템을 연결해야 하는 통신망 설계 등과 같은 곳에 사용될 수 있다.

그래프에서 순회를 하면 n - 1개의 간선을 이동하면서 모든 정점을 방문하게 되므로 순회 경로는 신장 트리가 된다. DFS를 이용해 생성된 신장 트리를 깊이 우선 신장 트리(depth first spanning tree), BFS를 이용해 생성된 신장 트리를 너비 우선 신장 트리(breadth first spanning tree)라 한다.

최소 비용 신장 트리

신장 트리 중 가중치의 합이 최소인 신장 트리를 최소 비용 신장 트리(minimum cost spanning tree)라 한다. 최소 비용 신장 트리를 만드는 방법으로 크루스칼 알고리즘과 프림 알고리즘이 있다.

크루스칼 알고리즘

크루스칼 알고리즘은 가중치가 낮은 간선을 삽입하면서 MST를 만드는 것이다.

참고자료

- 누구나 자료 구조와 알고리즘
- [무료] C로 배우는 자료구조 및 여러가지 예제 실습
- C로 배우는 쉬운 자료구조
- 코딩 테스트를 위한 자료 구조와 알고리즘 with C++
- 모두의 코드 씹어먹는 C++ - <10 - 1. C++ STL - 벡터(std::vector), 리스트(list), 데크(deque)>